CS271: Solving Phishing URLs Classification Problem With Machine Learning

Andrii Vlasiuk
Computer Science Department
San Jose State University
San Jose, CA 95192
408-924-1000
andrii.vlasiuk@sjsu.edu

ABSTRACT

Cybersecurity is one of the most popular topics, no matter the era. Attackers are trying to breach security and obtain sensitive information every day. Among different types of attacks, phishing is one of the most popular and widely used methods. It is a type of attack that involves sharing fake URLs and websites to trick victims into revealing sensitive information, including credentials, passwords, personal data, and financial details. For a long time, blacklisting suspicious websites was a common way to address phishing. However, it falls short because the method works only on websites that have already been identified as malicious. It lacks the ability to predict whether the website or URL is malicious before it can cause substantial damage. This paper treats phishing URL detection as a classification problem, explores how new methodologies, such as machine learning, have revolutionized the way phishing websites are detected, and research whether improving the basic machine learning model helps increase the accuracy of the predictions. The paper will provide background, explore what has already been studied, and identify a gap corresponding to the large difference between the approaches. The literature went from a simple machine learning approach to much more complex architecture, without mentioning the effects of improving the model itself. The project will then conduct an experiment to see whether staying within the same model and improving it would give a good enough boost to the model's performance.

1. INTRODUCTION

Phishing is one of the most common scams on the internet. It became even more common with the popularity of AI. In 2025, there were over 1 million phishing attacks each quarter, with nearly 90% of them utilizing AI. Criminals use deceptive URLs that they put into messages or emails to try to obtain victims' sensitive data like credentials, financial data, and other sensitive information. People are trying to protect the masses from falling prey to phishing attacks by publishing lists of malicious websites. However, it has one major flaw: it only helps against phishing websites that have already been identified and proven malicious. It means that somebody has already fallen into a trap before it could be identified. This is not a proper way to prevent attackers from successfully obtaining victims' sensitive information. If people want to prevent phishing attacks, those attacks have to be identified in real time. This is where AI comes in. Phishers use AI to run phishing scams, but that does not mean we cannot use AI to prevent them. In this project, I will explore how well a trained model can determine whether a URL is malicious. A lot of research shows that the models perform well on the URLs that are decisively malicious. One such system is PhishStorm, described in the *paper "PhishStorm: Detecting Phishing With Streaming Analytics*," which uses streaming analytics to determine whether a URL is safe. It was one of the first systems to detect malicious URLs in real time. This project's goal is to create something similar and run it on the dataset containing safe and malicious URLs.

Moreover, the goal is to investigate whether upgrading the model from a simple MLP to a more complex version brings sufficient improvements to model performance. Instead of introducing more complex concepts like the ones described in the next section, the project aims to improve the model itself. The baseline model is a simple MLP, and suggested improvements to the MLP include adding more hidden layers, normalization, and tuned dropout. This project aims to improve classification performance by increasing the model's complexity.

.

2. RESEARCH BACKGROUND

The existing research shows that phishing detection mechanisms have evolved from searching for websites in blacklists to machine learning and deep learning approaches. The research has also shown that phishing detection heavily relies on machine learning and deep learning methods, instead of searching the blacklists. Phishing attacks become more complicated, so the methods to detect them should move forward as well. The research shows multiple approaches for phishing detection. Some research papers prefer a classical machine learning approach. Papers such as *Phishing Website Detection Using Machine Learning* use machine learning to identify malicious websites [1]. The paper aims to improve classical machine learning algorithms without turning to deep learning, because deep learning requires significantly more resources. *An Efficient Phishing Attack Detection using Machine Learning Algorithms* and *Deep Learning for Phishing Website Detection* papers use different supervised learning algorithms, such as Random Forest and Support Vector, to classify URLs as either legitimate or phishing. Moreover, the two papers compare results, showing that Random Forest is more accurate than Support Vector [2], [3]. *A Lightweight Phishing Website Detection Algorithm by Machine Learning* proposed a lightweight

framework to efficiently detect malicious websites. First, the screening method is used to determine all phishing sites. After that, machine learning, specifically a CNN in this paper, is used to determine whether the website is malicious [4]. These three papers use classical machine learning methods to flag phishing websites. They use simple models and are successful nonetheless.

Some studies prioritize performance and try to combine different types of models and their respective benefits to improve the speed and accuracy. Some papers try to achieve a real-time performance. The *Real-Time Phishing Attack Detection through Advanced Machine Learning Techniques* paper uses the same machine learning algorithms mentioned above, as well as a Deep Neural Network, and then they collect the real-time data and features to achieve real-time phishing prediction. The paper calls their approach EnsemblePhish, since it combines multiple machine learning algorithms to enhance performance [5]. *Phishing Website Detection Based on Multidimensional Features Driven by Deep Learning* paper conducts a similar research by combining multiple feature views to improve detection accuracy and speed, and reduce the false positives. They also have an ensemble of multiple machine learning algorithms when using the multidimensional features [6].

Additionally, there are other papers that explore stronger deep learning methods to maximize phishing detection performance in terms of speed and accuracy. Then, some papers take it even further. *Intelligent Phishing Detection: A Machine Learning and Deep Learning Approach* paper provides a detailed comparison of machine learning algorithms like Logistic Regression, Decision Trees, ANNs, and so on. Moreover, the paper also aims to improve the results by combining the algorithms with boosting approaches like AdaBoost and Gradient Boosting, and showing that boosting results in significant performance improvement [7]. The *Phishing URL Detection Using Machine Learning and Deep Learning* paper dives into the comparison of machine learning and deep learning approaches to correctly classify the URLs. It concludes that deep learning models based on CNNs give a significant boost and achieve a nearly perfect result compared to machine learning models [8]. *A Survey of Intelligent Detection Designs of HTML URL Phishing Attacks* paper compares multiple deep learning models and how they deal with the classification problem for the URLs. It gives a direction to look into, concluding that using a deep learning model with the CNN is the way to go [9].

All of these papers prove that the most efficient way to detect phishing websites is by using machine learning methods and improving them by combining multiple models, increasing the performance, and introducing deep learning models. The research described in this section is the main motivation for this project. Rather than going from the baseline to a completely different type of model, this project moves from a simple model to a more complex one, but the model stays the same. The objective of the practical part is to see how much of a performance boost we can achieve by only improving the existing baseline model.

3. METHODOLOGY

3.1. PROBLEM SETUP

As mentioned before, the goal of this project is to represent phishing detection as a machine learning classification problem and solve it [1]. To solve this problem, we will train a model to classify URLs as legitimate or phishing and evaluate its accuracy. The practical part involves transitioning from the baseline, a simple MLP, to a more complex model with additional hidden layers, normalization, and dropout. Then we will compare the results of an improved model to the baseline to see whether we can achieve a significant boost in accuracy without involving more complex concepts. This will confirm that machine learning is a great way to solve the phishing detection problem and determine whether fine-tuning the model gives sufficient improvement [3].

3.2. DATASET

The dataset I've chosen for this project is called the Vrbančič Phishing Dataset. It is a relatively simple dataset consisting of 56,712 legitimate and 30,497 phishing URLs. Data Distribution is slightly unbalanced with 65%-35% relation. I chose this dataset because it is large and complex enough to provide meaningful results to both the baseline and improved model. The preprocessing steps would include removing duplicates and missing values, if present. Then, I would check which columns from the dataset I need and which ones I don't.

3.3. DATA PREPROCESSING

The Vrbančič Phishing Dataset will be split into three parts for training, to prevent overfitting. The ratio of proper and phishing URLs would be the same in each part. The split would be 70% for training, 15% for validation, and 15% for testing. The training set would be used to teach the model, the validation set would be used to check how well the model is performing, and the testing set would show how the model performs on the unseen data.

3.4. BASELINE MODEL

The baseline model that I will use for this project is a Multi-Layer Perceptron (MLP). This model is good for tabular datasets and is great for identifying nonlinearity in the website features. The simple MLP would serve as a baseline and would be compared against the proposed improvements later. Our model will have an input layer, two hidden layers, and an output layer for classification. The hidden layers will have 128 and 64 neurons. We will use ReLU as a non-linear activation function.

3.5. IMPROVEMENT

The improvement for this project is directly related to the model. The baseline MLP model will get enhanced with the goal of improving the classification performance without changing the dataset or the data within it. The simple MLP that is used as a baseline is straightforward, having just a couple of hidden layers. The improved model will include additional hidden layers, normalization, and a tuned dropout to improve the training and performance.

The improved MLP will have more hidden layers and use batch normalization between those hidden layers. The hidden layers will have different sizes, such as 256, 128, and 64. Each of the hidden layers will have a ReLU activation function. The improved model will also prevent overfitting by implementing the dropout tuning. These improvements are directly related to the model itself and will show how much of a boost the model can theoretically get.

3.6. TRAINING PROCEDURE

There are some similarities in the training procedure for the two models. The core training components are the same for both the simple MLP and the improved version. This way, the results that will come up after the training will be comparable. The models will use Binary Cross-Entropy Loss and the Adam Optimizer. The learning rate for the models will be 0.001, the batch size will be 64. Both models will be trained for 25 epochs with the possibility of early stopping if training does not give sufficient improvement. Both models will use the training set for learning and the validation set for testing.

Then, there are some differences between the baseline and the improved model related to training. The simple MLP will use only two hidden layers and have the ReLU activation function throughout the network. The improved model will have more hidden layers, batch normalization, and dropout. We will apply the batch normalization after the model passes through the hidden layers, which will stabilize the training process. We will add the dropout between the layers to prevent overfitting without losing the model's ability to learn. This way, we assume the model will train more efficiently, leading to faster, smoother convergence.

Moreover, as mentioned earlier, we will implement early stopping to prevent overfitting. If we notice that the results are not getting significantly better between the epochs, we stop the training process. This way, even if our two models will give similar results in the end, we might see a better performance if one of the models takes fewer epochs to achieve that result. The final result will determine how well each of the two models deals with the classification problem for legitimate and phishing URLs

3.7. SUMMARY DIAGRAM

Below is the summary diagram for the project structure, outlining the general idea of the project, showing the steps in which the project is executed, and highlighting the main details of each step.

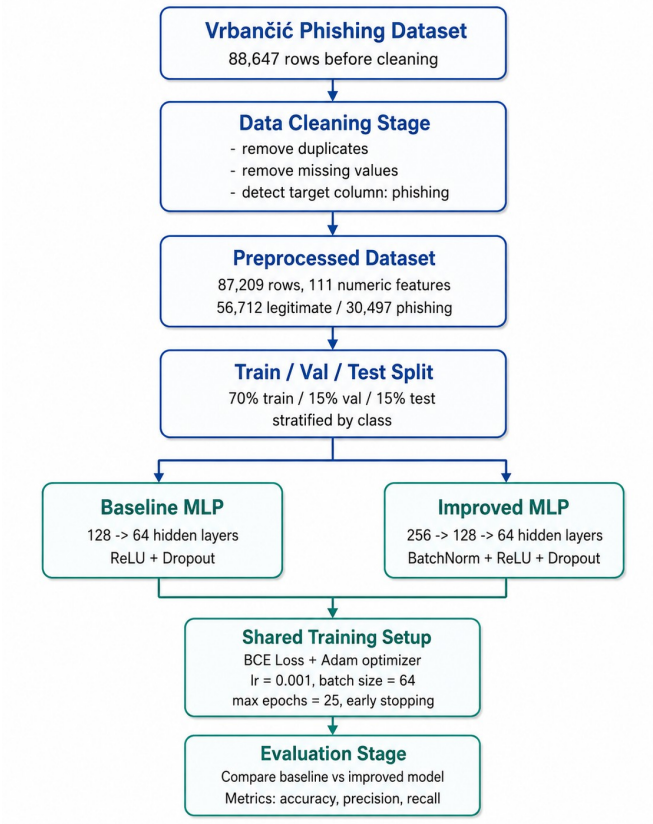

Figure 1. Summary Diagram

4. EXPERIMENTS

4.1. EXPERIMENTAL GOAL

The practical part of this project compares the two models and checks whether increasing the complexity of the model would improve the performance in the context of a classification problem, and if it does, whether the improvement is sufficient for the increased complexity [7].

4.2. EVALUATION METRICS

To keep the evaluation simple, the project uses three main metrics: Accuracy, Precision, and Recall. Accuracy measures the overall correctness of the models' predictions. Precision shows the percentage of phishing URLs among all URLs that the models classified as phishing. Finally, recall will show the percentage of phishing URLs successfully caught by the models.

4.3. EXPERIMENTAL PROCEDURE

The two models will train on the same dataset, with the same characteristics, including the training split and other settings. Moreover, the last comparison is also on the same test set for both models. This way, we remove any bias from the data that the two

models are trained on. By running the two models on the exact same data, including using the same data for the same part of training (training, validation, testing) we make sure they are in the same training conditions.

4.4. COMPARISON METRICS

The goal of the project is to determine whether the improved model achieves a better performance than the baseline. Therefore, the final step of the project is to compare the key metrics defined above (accuracy, precision, and recall) for the two models. These metrics will tell us which model solves the classification problem better and more safely. Moreover, by recording how many epochs each model took, we will also determine which model was faster when training.

5. RESULTS

After running the two models on the clean Vrbančič Phishing Dataset, without empty rows and duplicates, we obtained the results for the two models. Both the baseline and the improved model were trained using the same sets and with the same optimization settings. The results for the three metrics are outlined in the following table.

Model	Accuracy	Precision	Recall
Baseline MLP	0.9601	0.9504	0.9346
Improved MLP	0.9581	0.9340	0.9471

Table 1. Test Performance Comparison

From these results, we see that the performance of the two models is almost identical. The basic model has higher accuracy and precision compared to the improved model. In the meantime, the improved model has a higher recall. Generally speaking, it means that the baseline model is slightly better at overall prediction of whether the given URL is malicious or not, and at avoiding false positives. On the other hand, the improved model is slightly better at identifying the malicious URLs specifically, even though it results in some valid URLs being classified as phishing. We can say that the improved model is more sensitive to phishing patterns, so it will take the safer approach, which results in classifying some legitimate websites as potentially malicious.

These results conclude that increasing the model's complexity does not give a definitive improvement in solving the classification problem. It changed the approach that the model takes with regard to the problem, since the implemented improvements, such as additional hidden layers, batch normalization, and a dropout, increased the recall, but lost some accuracy. It might be useful depending on the type of model you want to end up with. If you would rather prioritize identifying all of the phishing URLs, even if it means that some legitimate ones might end up being marked as malicious, then using the improved model has its benefits. However, when it comes to overall performance, the suggested improvements are not sufficient. Therefore, taking one of the deeper and more complex approaches, like Decision Trees or deep learning with CNNs, is necessary to achieve the next level of performance [6]-[9].

6. CONCLUSION

This paper studied a classification problem of identifying legitimate and malicious URLs. We went over the history of identifying phishing URLs, which went from simply checking the blacklists for a specific website to utilizing the most recent methodologies like deep learning and CNNs. Then, we practically investigated whether machine learning does a good job at identifying Phishing URLs, and how much performance improvement we can achieve by making improvements to the baseline MLP. We tested the two models using the Vrbančič Phishing Dataset. We cleaned the dataset to get rid of empty rows and duplicates, and split it into training, validation, and testing parts.

The goal of this project was to determine whether improving the architecture of the baseline MLP would lead to sufficient improvement in performance, that is, whether or not the improved model would be more accurate and converge faster than the baseline. The central focus of the project was to improve the model itself, not create a new one or introduce a more complex architecture. The only changes we considered were the changes to already existing architecture, such as adding additional hidden layers, introducing batch normalization, and dropout. The results showed that the improved model's performance is not better than the baseline. The results are roughly equivalent for the three key metrics. The baseline has slightly higher accuracy and precision, while the improved model has higher recall.

Finally, the project shows that changing the baseline can change the direction of the model in terms of what it considers the success criteria, but it does not mean the model will get faster or more accurate. Therefore, if the goal is to increase the performance of the machine learning model, the next step would be to change the architecture, taking it to the next level involving methodologies such as CNNs.

7. REFERENCES

[1] A. Mandadi, S. Boppana, V. Ravella, and R. Kavitha, "Phishing Website Detection Using Machine Learning," in *2022 IEEE 7th International Conference for Convergence in Technology (I2CT)*, 2022.

[2] P. Chinnasamy, N. Kumaresan, R. Selvaraj, S. Dhanasekaran, K. Ramprathap, and S. Boddu, "An Efficient Phishing Attack Detection using Machine Learning Algorithms," in *2022 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC)*, 2022.

[3] K. S. N. Sushma, M. Jayalakshmi, and T. Guha, "Deep Learning for Phishing Website Detection," in *2022 IEEE 2nd Mysore Sub Section International Conference (MysuruCon)*, 2022.

[4] C. Gu, "A Lightweight Phishing Website Detection Algorithm by Machine Learning," in *2021 International Conference on Signal Processing and Machine Learning (CONF-SPML)*, 2021.

[5] H. Sharma, P. Sharma, and R. Singh, "Real-Time Phishing Attack Detection through Advanced Machine Learning Techniques," in *2023 IEEE International Conference on ICT in Business Industry & Government (ICTBIG)*, 2023.

[6] P. Yang, G. Zhao, and P. Zeng, "Phishing Website Detection Based on Multidimensional Features Driven by Deep Learning," *IEEE Access*, vol. 7, pp. 15196–15209, 2019.

[7] Y. V. S. Murthy, A. Gaur, S. Hiremath, and M. Hari Kishor, "Intelligent Phishing Detection: A Machine Learning and Deep Learning Approach," in *2025 IEEE 6th India Council International Subsections Conference (INDISCON)*, 2025.

[8] R. Ferdaws and N. E. Majd, "Phishing URL Detection Using Machine Learning and Deep Learning," in *2024 IEEE World AI IoT Congress (AIIoT)*, 2024.

[9] S. Asiri, Y. Xiao, S. Alzahrani, S. Li, and T. Li, "A Survey of Intelligent Detection Designs of HTML URL Phishing Attacks," *IEEE Access*, vol. 11, pp. 6421–6443, 2023.